

HotelOps

Hotel Management Demo

Project Explanation PDF

Prepared for a 50-60 page project explanation with index

Technology stack: Node.js, Express, EJS, ejs-mate, MongoDB, Mongoose, express-session, Bootstrap, and Font Awesome

Default server: <http://127.0.0.1:9999/>

Default database: `mongodb://127.0.0.1:27017/hotel_management`

Document scope

This PDF explains the purpose, architecture, files, routes, models, templates, data flow, admin workflow, limitations, and future enhancements of the HotelOps project.

Index / Table of Contents

Page 04	Project overview and purpose
Page 05	Project objectives
Page 06	Technology stack summary
Page 07	Repository structure
Page 08	Application entry point
Page 09	Database connection
Page 10	Express middleware pipeline
Page 11	Session management
Page 12	Admin authentication concept
Page 13	Authorization middleware
Page 14	Public home page
Page 15	Generated hotel image endpoint
Page 16	Contact page
Page 17	Admin dashboard route
Page 18	Room inventory model
Page 19	Booking model
Page 20	Admin user model
Page 21	Booking list page
Page 22	Create booking page
Page 23	Room locking during booking
Page 24	Checkout flow
Page 25	Logout behavior
Page 26	EJS layout strategy
Page 27	Navigation includes
Page 28	Admin page user experience
Page 29	Bookings page user experience
Page 30	New booking form experience

Index / Table of Contents

Page 31	Styling system
Page 32	Bootstrap and icon usage
Page 33	Demo data seeding
Page 34	Important application routes
Page 35	Request and response lifecycle
Page 36	Data flow for login
Page 37	Data flow for dashboard metrics
Page 38	Data flow for creating bookings
Page 39	Data flow for checkout
Page 40	Error handling approach
Page 41	Security observations
Page 42	Performance considerations
Page 43	Maintainability considerations
Page 44	Development setup
Page 45	MongoDB setup guidance
Page 46	Presentation talking points
Page 47	Code walkthrough order
Page 48	Strengths of the project
Page 49	Known limitations
Page 50	Future enhancement ideas
Page 51	Suggested database improvements
Page 52	Suggested UI improvements
Page 53	Suggested deployment plan
Page 54	Testing strategy
Page 55	Project conclusion
Page 56	Appendix: final summary

4. Project overview and purpose

Functionality overview (40%)

1. HotelOps is a hotel management demonstration web application focused on public hotel presentation and a protected administration workflow.
2. The project shows how a small Express application can combine server routes, reusable EJS layouts, MongoDB-backed data, session login, and polished Bootstrap styling.
3. It is suitable for college presentation, portfolio review, and as a foundation for adding production-ready hotel operations features.
4. The main business goal is to help staff view room inventory, understand occupancy, create bookings, and release rooms at checkout.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Project overview and purpose' behavior.

Technical code reference (60%)

```
const express = require("express");
let app = express();
const port = 9999;
app.listen(port, () => console.log(`listing on port ${port}...`));
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

5. Project objectives

Functionality overview (40%)

1. Provide a clear landing page for customers and visitors.
2. Allow only authenticated administrators to access internal room and booking information.
3. Persist rooms, bookings, and administrator users in MongoDB instead of storing them only in memory.
4. Demonstrate basic create and update operations for bookings and rooms through Mongoose models.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Project objectives' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

6. Technology stack summary

Functionality overview (40%)

1. The application uses Node.js, Express, EJS, ejs-mate, MongoDB, Mongoose, express-session, Bootstrap, and Font Awesome.
2. Express handles HTTP routing and middleware setup.
3. EJS renders dynamic HTML pages, while ejs-mate provides layout inheritance so common page structure is not repeated.
4. Mongoose defines schemas for administrator users, rooms, and bookings and connects those models to MongoDB collections.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Technology stack summary' behavior.

Technical code reference (60%)

```
app.engine("ejs", ejsMate);
app.set("view engine", "ejs");
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, "public")));
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

7. Repository structure

Functionality overview (40%)

1. app.js is the central application file where middleware, database connection, routes, authentication, and seeding behavior are configured.
2. models/ contains AdminUsers.js, Room.js, and Booking.js, which define the database shape for core entities.
3. views/ contains layout files, reusable navbar includes, and page-specific EJS templates.
4. public/home.css contains the custom visual language used by the public pages and the admin dashboard.
5. Security responsibility: protect admin-only URLs with session checks, validate user input on the server, and avoid exposing database details to the browser.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Repository structure' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

8. Application entry point

Functionality overview (40%)

1. The application starts from app.js and creates an Express server instance.
2. The configured port is 9999, so local users open the app through `http://127.0.0.1:9999/`.
3. The server configures EJS, static file serving, URL-encoded form parsing, and sessions before defining route handlers.
4. Database connection startup happens near the beginning so Mongoose models can be used by the routes.
5. Maintenance responsibility: keep repeated UI pieces in includes, keep static assets in public, and move larger route groups into modules when the application grows.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Application entry point' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

9. Database connection

Functionality overview (40%)

1. HotelOps connects to MongoDB using the local connection string `mongodb://127.0.0.1:27017/hotel_management`.
2. Mongoose provides the connection layer and converts JavaScript objects into MongoDB documents.
3. A successful connection message is printed in the terminal so the developer can confirm that the database is reachable.
4. If MongoDB is not running, the catch block logs the error, which helps diagnose local setup issues.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Database connection' behavior.

Technical code reference (60%)

```
async function main() {
  await mongoose.connect("mongodb://127.0.0.1:27017/hotel_management");
}
main().then(() => console.log("Database connection successfull"));
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

10. Express middleware pipeline

Functionality overview (40%)

1. `express.urlencoded` parses submitted HTML forms and exposes fields through `req.body`.
2. `express.static` serves CSS files and other assets from the public directory.
3. `express-session` stores the logged-in admin state between browser requests.
4. The app also imports `method-override`, which can be used when forms need to simulate HTTP verbs beyond GET and POST.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Express middleware pipeline' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

11. Session management

Functionality overview (40%)

1. The session middleware uses a secret, resave disabled, and saveUninitialized disabled.
2. The cookie is marked httpOnly so client-side JavaScript cannot read it directly.
3. sameSite lax provides baseline cross-site request protection for normal navigation flows.
4. The six-hour max age keeps the admin logged in for a practical work shift while still expiring inactive sessions.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Session management' behavior.

Technical code reference (60%)

```
app.use(session({
  secret: process.env.SESSION_SECRET || "dev_secret_change_me",
  resave: false, saveUninitialized: false,
  cookie: { httpOnly: true, sameSite: "lax", maxAge: 1000 * 60 * 60 * 6 },
}));
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

12. Admin authentication concept

Functionality overview (40%)

1. The login form posts a username and password to the server.
2. The server searches the admin user collection for the submitted username.
3. If the user is missing or the password does not match, the login template is rendered again with an error message.
4. When credentials are valid, req.session.isAdmin is set to true and the admin username is stored for display.
5. Security responsibility: protect admin-only URLs with session checks, validate user input on the server, and avoid exposing database details to the browser.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Admin authentication concept' behavior.

Technical code reference (60%)

```
const foundAdmin = await admin.findOne({ username: user });
if (!foundAdmin) return res.status(401).render("page/login_admin", loginError);
if (foundAdmin.password !== pass) return
res.status(401).render("page/login_admin", loginError);
req.session.isAdmin = true; req.session.adminUser = foundAdmin.username;
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

13. Authorization middleware

Functionality overview (40%)

1. The `requireAdmin` middleware checks whether the current request has an active administrator session.
2. Protected routes call `requireAdmin` before their controller logic runs.
3. If the user is authenticated, `next()` allows the request to continue.
4. If the user is not authenticated, the browser is redirected to `/login`.
5. Maintenance responsibility: keep repeated UI pieces in includes, keep static assets in public, and move larger route groups into modules when the application grows.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Authorization middleware' behavior.

Technical code reference (60%)

```
const requireAdmin = (req, res, next) => {
  if (req.session?.isAdmin) return next();
  return res.redirect("/login");
};
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

14. Public home page

Functionality overview (40%)

1. The home route renders the main public landing page.
2. The home page presents the hotel brand, feature highlights, room-oriented messaging, and call-to-action navigation.
3. It uses the shared public layout and navbar include for consistent structure.
4. Styling from public/home.css creates the modern visual theme with gradients, cards, rounded panels, and responsive spacing.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Public home page' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

15. Generated hotel image endpoint

Functionality overview (40%)

1. The /assets/hotel-image.svg endpoint generates an SVG image directly from server code.
2. Query parameters can influence the seed, width, and height while still being clamped to reasonable values.
3. A small hash function changes gradient hues based on the seed value.
4. The response is sent as image/svg+xml and can be displayed by the browser like a normal image asset.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Generated hotel image endpoint' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

16. Contact page

Functionality overview (40%)

1. The contact route renders a visitor-facing page for communication details and inquiries.
2. It fits into the public side of the site and uses the same visual system as the home page.
3. A contact page is useful in a hotel project because guest communication is a natural part of hotel operations.
4. The current version focuses on presentation rather than submitting messages to a database.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Contact page' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

17. Admin dashboard route

Functionality overview (40%)

1. The /admin route is protected by requireAdmin and loads room and booking data from MongoDB.
2. It fetches rooms sorted by room number and finds distinct booked room numbers from the bookings collection.
3. The route calculates total rooms, occupied rooms, available rooms, and occupancy percentage.
4. These statistics are passed to the admin dashboard template and shown as key performance indicators.
5. Security responsibility: protect admin-only URLs with session checks, validate user input on the server, and avoid exposing database details to the browser.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Admin dashboard route' behavior.

Technical code reference (60%)

```
const [rooms, bookedRoomNumbers] = await Promise.all([
  Room.find().sort({ room: 1 }).lean(),
  Booking.distinct("room"),
]);
const occupancyPct = rooms.length ? Math.round((occupiedRooms / rooms.length) *
100) : 0;
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

18. Room inventory model

Functionality overview (40%)

1. The Room model stores a room number, status, and price.
2. The room number is numeric and required, making sorting and matching straightforward.
3. The status field shows whether the room is available or occupied.
4. The price field supports displaying a nightly rate or operational charge for each room.
5. Maintenance responsibility: keep repeated UI pieces in includes, keep static assets in public, and move larger route groups into modules when the application grows.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Room inventory model' behavior.

Technical code reference (60%)

```
const roomSchema = new mongoose.Schema({
  room: { type: Number, required: true },
  status: { type: String, required: true },
  price: { type: Number, required: true }
});
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

19. Booking model

Functionality overview (40%)

1. The Booking model stores guest name, room number, contact number, check-in, and checkout values.
2. The name, room, and contact number fields are required to ensure the booking has essential guest information.
3. Check-in and checkout are currently stored as strings, which works for display but should become Date fields for advanced reporting.
4. Bookings provide the link between guests and room occupancy.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Booking model' behavior.

Technical code reference (60%)

```
const bookingSchema = new mongoose.Schema({
  name: { type: String, required: true }, room: { type: Number, required: true },
  contactno: { type: Number, required: true }, checkin: { type: String },
  checkout: { type: String }
});
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

20. Admin user model

Functionality overview (40%)

1. The AdminUsers model defines a username and password for administrators.
2. The username is required, unique, and capped at twenty-five characters.
3. The password is required and has a minimum length rule.
4. The model binds to a named MongoDB collection so login records can be managed separately from rooms and bookings.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Admin user model' behavior.

Technical code reference (60%)

```
const adminLoginSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true, maxLength: 25 },
  password: { type: String, required: true, minLength: 5 }
});
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

21. Booking list page

Functionality overview (40%)

1. The /admin/bookings route is protected and lists recent booking documents.
2. The query sorts records by descending identifier and limits the output to fifty rows.
3. The template shows guest names, contact numbers, room numbers, check-in values, checkout values, and actions.
4. A limited list keeps the page readable and avoids overwhelming the interface during demonstrations.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Booking list page' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

22. Create booking page

Functionality overview (40%)

1. The /admin/booking/new_booking route renders a protected booking form.
2. Administrators can enter guest name, contact number, room number, check-in date/time, and checkout date/time.
3. The post handler validates required fields before attempting to write to the database.
4. Room availability is checked and updated before the booking record is inserted.
5. Security responsibility: protect admin-only URLs with session checks, validate user input on the server, and avoid exposing database details to the browser.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Create booking page' behavior.

Technical code reference (60%)

```
const lockedRoom = await Room.findOneAndUpdate(
  { room, status: { $regex: /^available\s*$/i } },
  { $set: { status: "occupied" } },
  { returnDocument: "after" }
);
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

23. Room locking during booking

Functionality overview (40%)

1. The booking creation flow uses `findOneAndUpdate` to find an available room and mark it occupied.
2. The query matches the selected room number and checks availability with a case-insensitive regular expression.
3. If no matching available room is found, the route distinguishes between an invalid room and an unavailable room.
4. If booking insertion fails after the room update, the code sets the room back to available.
5. Maintenance responsibility: keep repeated UI pieces in includes, keep static assets in public, and move larger route groups into modules when the application grows.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Room locking during booking' behavior.

Technical code reference (60%)

```
try { await Booking.create({ name, contactno, room, checkin, checkout }); }
catch (err) {
  await Room.updateOne({ room }, { $set: { status: "available" } });
  throw err;
}
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

24. Checkout flow

Functionality overview (40%)

1. The checkout handler receives a booking id from the URL.
2. It validates the id as a MongoDB ObjectId before querying the booking collection.
3. After finding the booking, it extracts the room number and marks that room available again.
4. The booking is then deleted so the active booking list no longer shows that stay.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Checkout flow' behavior.

Technical code reference (60%)

```
if (!mongoose.Types.ObjectId.isValid(id)) return res.status(400).send("Invalid booking id.");
const booking = await Booking.findById(id).lean();
await Room.updateOne({ room: roomNo }, { $set: { status: "available" } });
await Booking.deleteOne({ _id: id });
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

25. Logout behavior

Functionality overview (40%)

1. The logout route destroys the current session.
2. If session destruction fails, the error is logged but the user is still redirected to the login page.
3. Logging out clears the administrator gate state and prevents further protected page access from the same session.
4. A logout link in the admin navigation makes this action easy to find.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Logout behavior' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

26. EJS layout strategy

Functionality overview (40%)

1. The application uses ejs-mate to support reusable layouts.
2. boilerplate.ejs provides the public page skeleton, while boilerplate_admin.ejs supports authenticated admin pages.
3. Reusable layout files keep head tags, CSS links, scripts, and common wrappers consistent.
4. Page templates focus on page-specific content instead of repeating the same HTML shell.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'EJS layout strategy' behavior.

Technical code reference (60%)

```
app.set("views", path.join(__dirname, "views"));
res.render("page/adminPage", { adminUser: req.session?.adminUser, stats, rooms });
<%- body %>
<%- include("../includes/navbarAdmin") %>
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

27. Navigation includes

Functionality overview (40%)

1. The includes folder stores navbar templates for public and admin pages.
2. The public navbar provides navigation to home, contact, and login-related destinations.
3. The admin navbar provides dashboard, booking, add-booking, and logout-oriented controls.
4. Keeping navigation in includes avoids duplicate markup across pages.
5. Security responsibility: protect admin-only URLs with session checks, validate user input on the server, and avoid exposing database details to the browser.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Navigation includes' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

28. Admin page user experience

Functionality overview (40%)

1. The dashboard presents high-level room statistics first, then detailed room information.
2. Key performance indicators help the administrator understand occupancy at a glance.
3. Room cards and tables visually separate available and occupied rooms.
4. The interface uses badges, icons, spacing, and contrast to make status information easy to scan.
5. Maintenance responsibility: keep repeated UI pieces in includes, keep static assets in public, and move larger route groups into modules when the application grows.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Admin page user experience' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

29. Bookings page user experience

Functionality overview (40%)

1. The bookings page focuses on recent guest stays and checkout actions.
2. It uses a table format because booking data is naturally structured into columns.
3. Clear action buttons help staff perform checkout without navigating through many screens.
4. Empty-state messaging can guide the admin when there are no current bookings.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Bookings page user experience' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

30. New booking form experience

Functionality overview (40%)

1. The new booking page keeps data entry fields simple and direct.
2. Each required field maps closely to a property on the Booking model.
3. The room number entry connects the guest booking to a room document.
4. Server-side validation protects the application even if a browser bypasses HTML validation.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'New booking form experience' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

31. Styling system

Functionality overview (40%)

1. The public/home.css file defines the visual language for the project.
2. It uses CSS variables for repeated colors, surfaces, shadows, muted text, and brand gold tones.
3. Large rounded containers and soft shadows create a hospitality-inspired presentation.
4. Responsive widths and grid-based sections help the pages adapt to different screen sizes.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Styling system' behavior.

Technical code reference (60%)

```
:root { --gold: #b98939; --surface: rgba(255, 255, 255, 0.78); }
.admin-shell .admin-kpi { display: flex; gap: 14px; align-items: center; }
.admin-shell .admin-panel { border-radius: 28px; box-shadow: 0 18px 44px rgba(55, 35, 10, 0.12); }
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

32. Bootstrap and icon usage

Functionality overview (40%)

1. Bootstrap is used for layout utilities, responsive grids, buttons, forms, tables, and spacing helpers.
2. Font Awesome icons improve visual recognition for navigation, statistics, and actions.
3. Using a framework reduces the amount of custom CSS needed for common interface patterns.
4. Custom CSS then adds brand-specific polish beyond the default Bootstrap appearance.
5. Security responsibility: protect admin-only URLs with session checks, validate user input on the server, and avoid exposing database details to the browser.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Bootstrap and icon usage' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

33. Demo data seeding

Functionality overview (40%)

1. The README explains that demo rooms and bookings are seeded once when collections are empty.
2. Seeding ensures the admin dashboard can show meaningful information during demonstrations.
3. Persistent data means the dashboard reflects MongoDB content instead of temporary in-memory arrays.
4. One-time seeding avoids replacing data every time the server restarts.
5. Maintenance responsibility: keep repeated UI pieces in includes, keep static assets in public, and move larger route groups into modules when the application grows.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Demo data seeding' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

34. Important application routes

Functionality overview (40%)

1. Important routes include `/`, `/contact`, `/login`, `/admin`, `/admin/bookings`, `/admin/booking/new_booking`, `/logout`, and `/assets/hotel-image.svg`.
2. GET routes render pages or assets, while POST routes process form submissions and state-changing actions.
3. Admin routes use the `requireAdmin` middleware to protect sensitive operations.
4. Public routes remain accessible so visitors can browse the site before login.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Important application routes' behavior.

Technical code reference (60%)

```
app.get("/", (req, res) => res.render("page/home"));
app.get("/login", (req, res) => res.render("page/login_admin", { error: null,
username: "" }));
app.get("/admin", requireAdmin, async (req, res) => { /* dashboard */ });
app.post("/admin/booking/new_booking", requireAdmin, async (req, res) => { /*
create */ });
// Supporting implementation pattern used in HotelOps
const payload = { adminUser: req.session?.adminUser };
const query = Model.find().sort({ _id: -1 }).lean();
const records = await query;
return res.render("page/template", { ...payload, records });
// Errors are handled by route-level try/catch blocks.
// Production improvement: move this logic into a route module or service.
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

35. Request and response lifecycle

Functionality overview (40%)

1. A browser request enters the Express middleware pipeline first.
2. Static files may be served immediately if the URL matches a file in public.
3. Page routes run controller logic, query models if needed, and render EJS templates.
4. Form submissions are parsed by `express.urlencoded` and then validated by route handlers.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Request and response lifecycle' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

36. Data flow for login

Functionality overview (40%)

1. The admin enters credentials on the login page.
2. The browser sends a POST request to /login with username and password fields.
3. The server queries the admin user model by username and compares the submitted password.
4. On success, session values are written and the browser is redirected to /admin.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Data flow for login' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

37. Data flow for dashboard metrics

Functionality overview (40%)

1. The dashboard route queries all rooms and distinct room numbers from bookings.
2. It converts booking room values to numbers and filters out invalid results.
3. A Set is used so duplicate booked room entries do not inflate occupied room count.
4. Available rooms are calculated as total rooms minus occupied rooms.
5. Security responsibility: protect admin-only URLs with session checks, validate user input on the server, and avoid exposing database details to the browser.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Data flow for dashboard metrics' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

38. Data flow for creating bookings

Functionality overview (40%)

1. The admin submits guest and stay details from the booking form.
2. The server validates each required value before writing to MongoDB.
3. A room is atomically changed from available to occupied through Mongoose.
4. A booking document is created after the room is successfully locked.
5. Maintenance responsibility: keep repeated UI pieces in includes, keep static assets in public, and move larger route groups into modules when the application grows.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Data flow for creating bookings' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

39. Data flow for checkout

Functionality overview (40%)

1. The admin clicks a checkout action on a booking record.
2. The server validates the booking identifier and loads the booking document.
3. The related room is set back to available.
4. The booking record is deleted after the room update succeeds.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Data flow for checkout' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

40. Error handling approach

Functionality overview (40%)

1. The app uses try/catch blocks around database operations in route handlers.
2. Server errors are logged to the console for developer visibility.
3. Validation errors return clear messages such as missing guest name, invalid contact number, or invalid room.
4. Authentication failures use generic invalid-credential text so attackers do not learn which usernames exist.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Error handling approach' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

41. Security observations

Functionality overview (40%)

1. The project demonstrates protected routes and session cookies, which are important security building blocks.
2. The session cookie uses httpOnly and sameSite settings.
3. The current password comparison uses plain text and should be replaced with bcrypt hashing for real systems.
4. The session secret should come from a secure environment variable outside source control.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Security observations' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

42. Performance considerations

Functionality overview (40%)

1. The app is lightweight and suitable for local demonstration workloads.
2. Dashboard queries are simple, but large hotels would need indexes, pagination, and aggregation pipelines.
3. The bookings route limits records to fifty, which protects the table from growing too large during normal use.
4. Static CSS is served directly by Express, which is acceptable for development and small demos.
5. Security responsibility: protect admin-only URLs with session checks, validate user input on the server, and avoid exposing database details to the browser.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Performance considerations' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

43. Maintainability considerations

Functionality overview (40%)

1. The project separates database models from views and public assets.
2. The central app.js file is easy to follow for a small project, but routes could be split into modules as the app grows.
3. Reusable layouts and navbar includes reduce duplicated template code.
4. Keeping README instructions current helps new developers run the app quickly.
5. Maintenance responsibility: keep repeated UI pieces in includes, keep static assets in public, and move larger route groups into modules when the application grows.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Maintainability considerations' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

44. Development setup

Functionality overview (40%)

1. Developers need Node.js, npm, and a local MongoDB server.
2. npm install installs dependencies listed in package.json and package-lock.json.
3. node app.js starts the Express server.
4. The home page is available at `http://127.0.0.1:9999/` and the login page is available at `/login`.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Development setup' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

45. MongoDB setup guidance

Functionality overview (40%)

1. The database name used by default is hotel_management.
2. Administrator login records must be inserted into the collection used by the Admin model.
3. Room and booking collections are managed by their Mongoose models.
4. Running MongoDB locally is essential because route handlers depend on database queries.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'MongoDB setup guidance' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

46. Presentation talking points

Functionality overview (40%)

1. Begin by explaining the problem: hotels need a quick way to manage rooms and bookings.
2. Show the public home page to demonstrate customer-facing branding.
3. Log in as an administrator to demonstrate protected access.
4. Review the dashboard metrics, booking list, new booking form, and checkout behavior.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Presentation talking points' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

47. Code walkthrough order

Functionality overview (40%)

1. Start with package.json to explain dependencies.
2. Open app.js to discuss imports, database connection, middleware, session setup, and routes.
3. Open the models folder to explain schemas and collections.
4. Open the views folder to show how EJS pages are rendered.
5. Security responsibility: protect admin-only URLs with session checks, validate user input on the server, and avoid exposing database details to the browser.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Code walkthrough order' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

48. Strengths of the project

Functionality overview (40%)

1. The project has a clear business theme that is easy for reviewers to understand.
2. It demonstrates both public and private web application areas.
3. It uses persistent database models instead of only static HTML.
4. It includes real operational actions such as creating a booking and checking out a guest.
5. Maintenance responsibility: keep repeated UI pieces in includes, keep static assets in public, and move larger route groups into modules when the application grows.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Strengths of the project' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

49. Known limitations

Functionality overview (40%)

1. Passwords are stored and compared as plain text in the current educational version.
2. Check-in and checkout are stored as strings instead of Date values.
3. There is no role hierarchy, so all authenticated admins have the same permissions.
4. There is no payment, invoice, housekeeping, or inventory module yet.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Known limitations' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

50. Future enhancement ideas

Functionality overview (40%)

1. Add password hashing with bcrypt and a secure registration or admin-management flow.
2. Add room categories such as standard, deluxe, suite, and family room.
3. Add booking date conflict checks so the same room can be reserved for non-overlapping future stays.
4. Add reports for occupancy trends, revenue, and monthly booking counts.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Future enhancement ideas' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

51. Suggested database improvements

Functionality overview (40%)

1. Add unique indexes for room numbers and administrator usernames.
2. Use Date fields for check-in and checkout so the system can compare and sort by time.
3. Store booking status values such as active, completed, cancelled, and no-show.
4. Add createdAt and updatedAt timestamps to models for auditing.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Suggested database improvements' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

52. Suggested UI improvements

Functionality overview (40%)

1. Replace free-text room entry with a dropdown of available rooms.
2. Add toast or flash messages after successful booking and checkout actions.
3. Add filters for room status, price range, and booking date.
4. Add confirmation dialogs before checkout or destructive actions.
5. Security responsibility: protect admin-only URLs with session checks, validate user input on the server, and avoid exposing database details to the browser.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Suggested UI improvements' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

53. Suggested deployment plan

Functionality overview (40%)

1. Move configuration values such as port, database URL, and session secret into environment variables.
2. Use a managed MongoDB service or a secured database server for production data.
3. Run the Node.js application behind a process manager such as PM2 or a container platform.
4. Serve the site over HTTPS and configure secure cookies in production.
5. Maintenance responsibility: keep repeated UI pieces in includes, keep static assets in public, and move larger route groups into modules when the application grows.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Suggested deployment plan' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

54. Testing strategy

Functionality overview (40%)

1. Start with syntax checks for JavaScript files to catch parsing errors.
2. Add unit tests for utility logic and validation functions after routes are modularized.
3. Add integration tests for login, protected route redirects, booking creation, and checkout.
4. Use a test MongoDB database so automated tests do not modify development data.
5. Controller responsibility: keep request parsing, validation, model calls, and response rendering in a predictable order so the route can be explained line by line.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Testing strategy' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

55. Project conclusion

Functionality overview (40%)

1. HotelOps is a complete educational demonstration of a small hotel management web application.
2. It combines routing, templates, sessions, database models, dashboard metrics, booking operations, and custom styling.
3. The codebase is simple enough to explain but realistic enough to show meaningful business workflow.
4. The project can be expanded into a stronger system by improving security, validation, reporting, and deployment practices.
5. Data responsibility: use Mongoose schemas to document which fields exist, which values are required, and which collection stores each business object.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Project conclusion' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.

56. Appendix: final summary

Functionality overview (40%)

1. HotelOps demonstrates a clear hotel workflow.
2. It uses a realistic full-stack JavaScript architecture.
3. It separates models, views, public assets, and route logic.
4. It includes protected administration pages.
5. View responsibility: use EJS templates and shared layouts so HTML structure is reusable while each page still receives dynamic data from Express.
6. Implementation focus: identify the route, schema, template, or stylesheet that owns the 'Appendix: final summary' behavior.

Technical code reference (60%)

```
// Typical HotelOps protected route pattern
app.get("/admin/example", requireAdmin, async (req, res) => {
  try {
    const records = await Model.find().sort({ _id: -1 }).lean();
    return res.render("page/template", {
      adminUser: req.session?.adminUser,
      records,
    });
  } catch (err) {
    console.error("Route error:", err);
    return res.status(500).send("Failed to load page.");
  }
});
```

Code-to-functionality mapping

- Route/middleware code controls when this feature runs and whether the admin session is required.
- Mongoose model code defines the data structure and performs the database read, create, update, or delete operation.
- EJS/template code receives server variables and converts the result into a page that users can understand.